

StreamingVLA: Streaming Vision-Language-Action Model with Action Flow Matching and Adaptive Early Observation

Supplementary Materials

Anonymous ECCV 2026 Submission

Paper ID #7649

A Real-World Robot Deployment

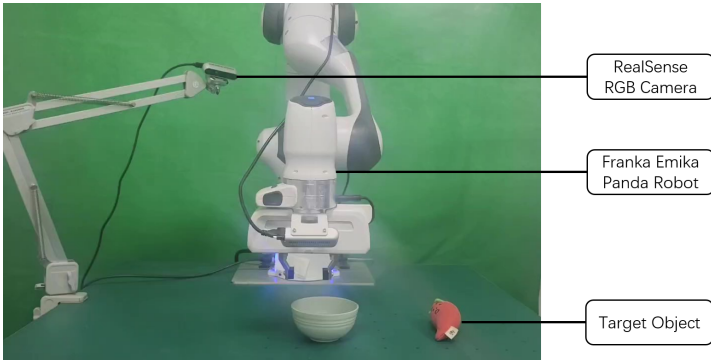


Fig. 1: Real-world Setup. The real-world evaluation platform includes a tabletop workspace, a Franka Panda robotic manipulator fixed to the table, and a RGB camera for visual observation. All objects are positioned on the table surface.

To validate the practical efficiency of our method, we deploy the model on a real-world robotic manipulation task. The task requires a robotic arm to pick up an object from a designated location and place it at a target position. This pick-and-place scenario represents a common manipulation setup that requires precise action execution and timely perception updates.

Our model is trained based on the $\pi_{0.5}$ -**base** architecture with an action horizon of $H = 8$. During deployment, we compare our **StreamingVLA** framework with the original $\pi_{0.5}$ policy under the same hardware setup. For the baseline configuration, the $\pi_{0.5}$ policy operates with an action horizon of $H = 8$ and a replanning step of 4 actions, while **StreamingVLA** uses the same horizon but performs streaming action generation.

Algorithm 1 Training algorithm

Require: Preprocessed training set $\mathcal{D} = \{(o_i, \alpha_i, \xi_i)\}_{i=1}^N$, horizon h , noise scale σ_0 , stabilizing factor k

- 1: **while** not converged **do**
- 2: $(o_i, \alpha_i, \xi_i) \sim \mathcal{D}$
- 3: Compute action state $\mathcal{A}_i$:
- 4: $\mathcal{A}_i[n] \leftarrow \mathcal{A}_i[n-1] + \xi_i[n-1]$, $\mathcal{A}_i[0] \leftarrow \alpha_i$
- 5: $t \sim \text{Uniform}(0, 1)$, index $T \leftarrow \lfloor t \cdot h \rfloor$
- 6: $x_T \sim \mathcal{N}(\mathcal{A}_i[T], \sigma_0^2 e^{-2kT/h})$
- 7: $\theta \leftarrow \theta - \lambda \nabla_{\theta} \underbrace{\|v_{\xi}(x_T, T/h) - v_{\theta}(x_T, T/h|o_i)\|^2}_{\text{Conditional flow matching loss}}$
- 8: **end while**
- 9: **return** v_{θ}

Both methods are deployed on the same robotic platform as Fig. 1 to ensure a fair comparison. The system consists of a table-top workspace with a Franka Emika Panda robot and a fixed RGB camera, whose captured images serve as visual input to the VLA model. The robot has seven actuated joints and a parallel-jaw gripper with open/close capability. We measure the average time per action demonstrated in Sec. 5 during the manipulation process. The results show that **StreamingVLA** achieves an average latency of **170.88 ms** per action, while the original $\pi_{0.5}$ baseline requires **271.49 ms** per action.

These results demonstrate that **StreamingVLA** significantly improves real-world control efficiency by reducing action generation latency while maintaining the same action horizon. The reduced latency enables faster response during execution, which is critical for real-time robotic manipulation. **A demonstration video of the real-world experiments is provided in the supplementary material.**

B Algorithms

We introduce the action-space state α as a new type of input during training. These states are directly read from the preprocessed dataset and used as the initial action state α_i of each sub-trajectory ξ_i , from which the ground-truth action states of the entire sub-trajectory $\mathcal{A}_i$ are computed. Consequently, our training algorithm differs from that of the streaming flow policy or $\pi_{0.5}$; We present the detailed description in Algorithm 1 as follows:

During inference we also need to maintain the action state, which is updated in real time according to the actions that have been executed. The corresponding procedure is described in Algorithm 2.

Algorithm 2 Inference algorithm

Require: $v_\theta(x_T, T/h|o)$, Early observation step N_{eo} , Observation indicator I with threshold η

```

1:  $o \leftarrow o_0, \alpha \leftarrow \mathbf{0}, T \leftarrow 0, \text{need\_obs} \leftarrow \text{True}$  //initialization
2: while True do
3:   if need_obs then
4:      $o \leftarrow \text{Observation}$  //async process observation
5:     need_obs  $\leftarrow$  False
6:   end if
7:   action  $a \leftarrow v_\theta(\alpha, T/h|o)^{\frac{1}{h}}$  //async generate action
8:   if  $h - T = N_{oe}$  then
9:     if  $I(a) \leq \eta$  then //adaptive early observation
10:      need_obs  $\leftarrow$  True
11:    end if
12:  end if
13:  if  $h - T = 0$  then
14:    need_obs  $\leftarrow$  True
15:     $T \leftarrow 0$ 
16:  end if
17:  execute( $a$ ) //async execute action
18:   $\alpha \leftarrow \alpha + a$  //update action state
19:   $T \leftarrow T + 1$ 
20: end while

```

C Additional Experimental Results**C.1 Runtime Breakdown**

We further analyze the runtime composition based on actual profiling using the analysis framework discussed in Sec. 3 and illustrated in Fig. 2, to evaluate the baseline method and identify the sources of the speedup in StreamingVLA.

For $\pi_{0.5}$, no acceleration methods are applied thus $O_{ge} = O_{oe} = 0$, the latency and halting time could be calculated as follows, matching the measured latencies:

$$T_{\text{action1}} = \frac{T_o + T_g + h_1 T_e}{h_1} = 74.6\text{ms},$$

$$T_{\text{halt1}} = T_o + T_g = 238\text{ms}.$$

For **StreamingVLA**, The overlap time O_{ge} and O_{oe} can be calculated as below:

$$O_{ge} = (h - 1) \min \left\{ \frac{T_g}{h}, \frac{T_e}{h} \right\} = 162\text{ms},$$

$$O_{oe} = N_{eo} \frac{T_e}{h} = 41.58\text{ms}.$$

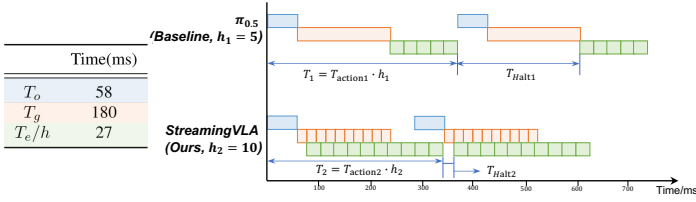


Fig. 2: The runtime breakdown based on actual profiling of $\pi_{0.5}$ and StreamingVLA. The three colored blocks correspond to the “observation”, “gen-action” and “execution” stage in the VLA workflow. The length of each block reflects the measured time.

Then we obtain:

$$T_{\text{action2}} = \frac{T_o + T_g + T_e - O_{ge} - O_{oe}}{h} = 30.44\text{ms},$$

$$T_{\text{halt2}} = T_o + T_g - (O_{ge} + O_{oe}) = 34.42\text{ms}.$$

In summary, StreamingVLA delivers a $2.40\times$ latency and $6.71\times$ halting gap improvements.

C.2 Design Details of Early Observation Predictor

The architecture of our predictor model is illustrated in Fig. 3. We utilized two NVIDIA A800-SXM4-80GB GPUs to train the predictor, taking 3-4 GPU hours to finish. During training, we randomly sample frame pairs with a temporal gap of 1–3 actions. A visual encoder first maps these observations into an embedding space. The predictor then takes the early frame embedding as input and the pending action sequence as condition to generate a residual embedding, $\Delta\text{embedding}$. The model is optimized using a Mean Squared Error (MSE) loss between the latent representation of the late frame and the predicted composite ($\text{embedding}_{\text{early}} + \Delta\text{embedding}$).

During inference, the L_2 -norm of the predicted $\Delta\text{embedding}$ serves as the metric for visual variance. Early observation is inhibited if this value exceeds a predefined threshold, and we select this threshold by analyzing the accuracy-latency tradeoff across various settings.

In our framework, each observation generates a horizon of H actions ($H = 10$ in our setting), which are appended to the pending execution queue. When the queue reduces to n remaining actions (a predefined parameter), the current observation and residual actions are transmitted to the server. The predictor in the server then evaluates whether to trigger an early observation.

Consequently, the predictor operates once per H actions, introducing a marginal latency of 8–10 ms per invocation. This translates to an average computational overhead of approximately 0.8–1 ms per action, representing only 5% of the standard generation time (18 ms). Despite this minimal overhead, our method

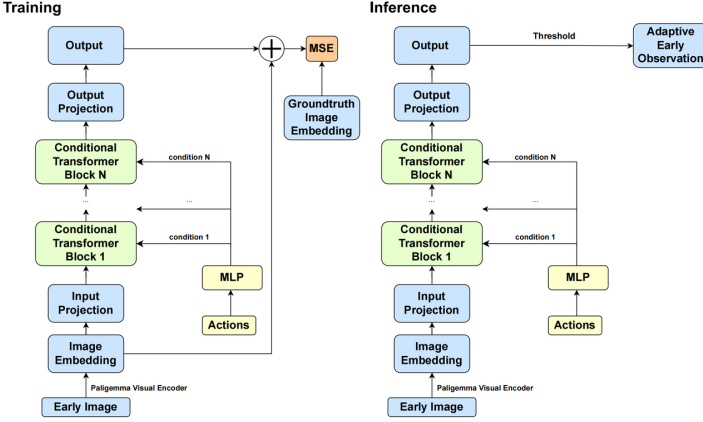


Fig. 3: Architecture of the predictor for adaptive early observation. During training, the model receives the early-frame embedding and the pending action sequence as conditions and predicts a residual embedding $\Delta embedding$ that estimates the latent change toward the future frame. The model is trained using an MSE loss between the ground-truth late-frame embedding and the reconstructed representation ($embedding_{early} + \Delta embedding$). At inference time, the L_2 norm of $\Delta embedding$ serves as a visual-variance indicator, which is compared with a threshold to determine whether early observation should be triggered.

achieves a significant accuracy improvement of 8.7% compared to the naive early observation baseline.

C.3 Visualization

We visualized representative tasks from each suite to provide an intuitive comparison of task completion speed and fluency between our method and the baselines. For each suite, we selected one representative task for visualization. The tasks corresponding to the four suites are as follows:

- **LIBERO-Spatial:** pick up the black bowl in the top drawer of the wooden cabinet and place it on the plate.
- **LIBERO-Object:** pick up the orange juice and place it in the basket.
- **LIBERO-Goal:** open the top drawer and put the bowl inside.
- **LIBERO-Long:** put both the alphabet soup and the tomato sauce in the basket.

We recorded the simulation state every 30 ms (approximately the time required to execute one action step in the LIBERO environment), and used these snapshots to generate video results. A subset of these results is shown in Fig. 4. During the “open the top drawer and put the bowl inside” task, the robotic arm controlled by the $\pi_{0.5}$ model experienced 12 pauses while opening the drawer, and even failed to open it once due to a prolonged pause. When picking up

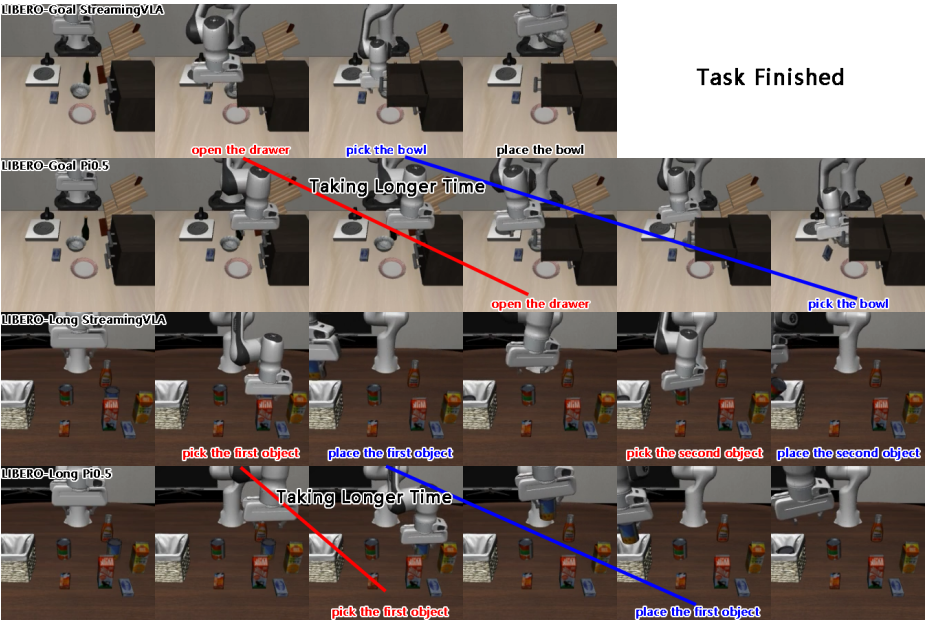


Fig. 4: Screenshots of our video simulation results. The images in each row are spaced 4 seconds apart. **Top two rows:** *open the top drawer and put the bowl inside.* StreamingVLA completes the task in under 16 seconds, whereas pi0.5 has not yet picked up the bowl at the 20-second mark. **Bottom two rows:** *put both the alphabet soup and the tomato sauce in the basket.* At 20 seconds, StreamingVLA has already finished the pick-and-place operations for both objects, while pi0.5 is still on its way to picking up the second item.

the bowl and placing it inside the drawer, the arm encountered an additional 25 pauses. These interruptions significantly delayed the task completion and even hindered its success. In contrast, the robotic arm controlled by StreamingVLA exhibited no noticeable halting throughout the execution of this task. The full visualization results, named “pi0.5_goal_task3_success.mp4” and “streamingVLA_goal_task3_success.mp4”, are included in the *Simulation Visualization Results* section of the supplementary materials, where we also provide more detailed visualization results.